

AlphaMinus: A Clean PyTorch Implementation of AlphaZero

Applied to Connect4 via Self-Play and Monte Carlo Tree Search

Vaishnavi Jha

Vellore Institute of Technology

Vaishnavi7a7@gmail.com

github.com/VJ70/AlphaMinus

Abstract

We present AlphaMinus, a complete, from-scratch PyTorch reimplementation of the AlphaZero algorithm applied to the game of Connect4. The system couples Monte Carlo Tree Search (MCTS) with a residual neural network that jointly predicts move probabilities and game outcomes, and trains exclusively through self-play with no human game data. After 30 training iterations comprising approximately 1,500 self-play games, AlphaMinus achieves a win rate of $\sim 95\%$ against a random agent and $\sim 80\%$ against a greedy baseline, with an estimated Elo improvement of +600 relative to random play. The codebase is fully documented, test-covered, and designed to be straightforwardly extensible to other board games. All code is available at <https://github.com/VJ70/AlphaMinus>.

1 Introduction

AlphaZero [1] demonstrated that a single reinforcement learning algorithm, given only the rules of a game, could reach or surpass human-level performance in Chess, Shogi, and Go. Its key insight is a closed self-improvement loop: a neural network guides Monte Carlo Tree Search, and the outcomes of those searches provide training signal for the network, iteratively bootstrapping stronger play without any human knowledge or hand-crafted features.

Despite the conceptual elegance of AlphaZero, public reimplementations are often incomplete, tightly coupled to specific frameworks, or difficult to understand and extend. AlphaMinus aims to fill this gap by providing:

- A minimal yet complete implementation of the full AlphaZero pipeline in PyTorch.
- A clean, modular architecture separated into game logic, MCTS, network, training, and evaluation components.
- Reproducible empirical results on Connect4 with documented hyperparameters.
- An extensible design that makes it straightforward to swap in other two-player zero-sum games.

Connect4 is a natural testbed: its 6×7 board is small enough that training converges in hours on a single GPU, yet the game is non-trivial and has a known optimal solution that provides an implicit ceiling against which learned agents can be benchmarked.

2 Background

2.1 Monte Carlo Tree Search

Monte Carlo Tree Search [5, 6] is a best-first search algorithm that grows a game tree by repeatedly executing four phases: selection, expansion, simulation, and backpropagation. AlphaZero replaces the rollout simulation with a neural network evaluation, dramatically improving sample efficiency.

2.2 AlphaZero

Silver et al. [1] train a single network θ that maps a board state s to a pair (p, v) , where p is a probability vector over legal moves and $v \in [-1, 1]$ is an estimate of the expected game outcome from the current player’s perspective. The network is used inside MCTS via the PUCT selection rule (see Section 3.3), and trained on self-play data using a combined policy and value loss (see Section 3.5).

3 System Design

3.1 Game Environment

The Connect4 environment exposes a standard interface: board state encoding, legal move enumeration, move application, and win/draw detection.

State Encoding. Each board position is encoded as an $8 \times 6 \times 7$ tensor:

Plane	Content
0	Current player’s pieces
1	Opponent’s pieces
2–6	Reserved for move history
7	Side to move (constant 0 or 1)

This encoding is deliberately game-agnostic in structure, following the convention used in the original AlphaZero paper.

3.2 Neural Network Architecture

The network θ is a residual neural network with a shared trunk and two output heads. Each ResBlock consists of two 3×3 convolutional layers with 128 filters, batch normalisation, and ReLU activations, with a skip connection that adds the block input to its output. The policy head applies a 1×1 convolution followed by a fully connected layer and log-softmax to produce log-probabilities over the 7 columns. The value head applies a 1×1 convolution, a fully connected hidden layer, and tanh to produce a scalar in $[-1, 1]$. Default hyperparameters are 6 residual blocks and 128 filters per layer.

3.3 Monte Carlo Tree Search

At each move, MCTS runs N_{sim} simulations from the current root state (default $N_{\text{sim}} = 200$).

Selection. Each simulation traverses the tree using the PUCT (Predictor + UCT) rule:

$$UCB(s, a) = Q(s, a) + c_{\text{puct}} \cdot P(s, a) \cdot \sqrt{N(s)} / (1 + N(s, a))$$

where $Q(s, a)$ is the mean value of node (s, a) , $P(s, a)$ is the prior probability from the network, $N(s)$ is the parent visit count, $N(s, a)$ is the child visit count, and $c_{\text{puct}} = 1.25$ controls the exploration–exploitation trade-off.

Expansion. When an unvisited leaf node is reached, the network is queried to obtain priors p and a leaf value v . All legal children are added to the tree with prior probabilities from p .

Backpropagation. The value v is propagated back up the path. Signs are flipped at each level since Connect4 is a zero-sum game and the parent evaluates the position from the opponent’s perspective.

Dirichlet noise. At the root node, Dirichlet noise with $\alpha = 0.3$ is mixed with the network priors (fraction $\epsilon = 0.25$) to ensure sufficient exploration during self-play.

Move selection. After all simulations, the policy target π is computed from visit counts:

$$\pi(a | s) \propto N(s, a)^{1/\tau}$$

where temperature τ controls exploration ($\tau = 1$ early in games, $\tau \rightarrow 0$ for greedy play in evaluation).

3.4 Self-Play Data Generation

Each self-play game produces a sequence of tuples (s_t, π_t, z) , where s_t is the board state at step t , π_t is the MCTS visit-count distribution, and $z \in \{-1, 0, +1\}$ is the game outcome from the perspective of the player to move at step t . Tuples are stored in a circular replay buffer of fixed capacity.

3.5 Training Procedure

Mini-batches of (s, π, z) tuples are sampled uniformly from the replay buffer. The network is trained to minimise the combined policy–value loss:

$$L(\theta) = -\sum_a \pi(a) \log p\theta(a | s) + (z - v\theta(s))^2$$

The first term is the cross-entropy between the MCTS policy target and the network’s predicted policy; the second is the mean-squared error between the game outcome and the network’s value prediction. Parameters are updated with Adam ($\eta = 10^{-3}$, batch size 512) with gradient clipping to stabilise training. Model checkpoints are saved after each training iteration.

4 Experimental Setup

4.1 Hyperparameters

Parameter	Default	Description
mcts.simulations	200	MCTS rollouts per move
mcts.c_puct	1.25	Exploration constant
mcts.dirichlet_alpha	0.3	Dirichlet noise concentration
mcts.dirichlet_epsilon	0.25	Noise fraction at root
network.n_res_blocks	6	Residual blocks in trunk
network.n_filters	128	Conv filters per layer
training.self_play_games	50	Games generated per iteration
optimizer.lr	10^{-3}	Adam learning rate
optimizer.batch_size	512	Training mini-batch size

All hyperparameters are specified in config.yaml and can be overridden from the command line.

4.2 Baselines

We evaluate against two fixed-policy agents:

- Random: selects uniformly at random from legal moves.
- Greedy: plays the move that immediately wins if available; otherwise blocks the opponent’s immediate win; otherwise plays randomly.

Elo ratings are estimated relative to the random agent, assigned an Elo of 0.

4.3 Training Setup

Training runs for 30 iterations. Each iteration generates 50 self-play games, adds their (s, π, z) tuples to the replay buffer, samples mini-batches for network updates, and evaluates the current agent against both baselines over 50 arena matches. Training curves (policy loss, value loss, win rates) are logged and plotted after each evaluation round.

5 Results

After 30 training iterations ($\approx 1,500$ self-play games):

Metric	Value
Win rate vs. Random agent	$\sim 95\%$
Win rate vs. Greedy agent	$\sim 80\%$
Estimated Elo (Random = 0)	+600

These results demonstrate that AlphaMinus successfully learns a strong Connect4 policy from pure self-play within a modest compute budget. The gap between performance against the random agent and the greedy agent is expected: the greedy agent’s one-step lookahead is a non-trivial heuristic that the learned policy must overcome by planning multiple moves ahead.

Training curves saved to `results/plots/training_curves.png` show monotonic improvement in both policy accuracy and value calibration over the course of training, with win rates increasing steeply in the first 10 iterations before levelling off.

6 Discussion

Extensibility. The MCTS and network components are game-agnostic; only the `Connect4` class in `game/connect4.py` is game-specific. Replacing it with an Othello or Gomoku environment requires no changes to the search or training code.

Scalability. The current implementation runs self-play games sequentially. Parallel self-play using Python multiprocessing is a natural next step that would substantially increase data throughput. Scaling the network to 19 residual blocks and 256 filters would bring it closer to the full AlphaZero scale used for Chess and Go.

History planes. The current state encoding reserves planes 2–6 for move history but does not populate them. Filling these with the previous three board states for each player would give the network the ability to detect repetitions and reason about game dynamics over time.

Temperature scheduling. A curriculum that starts with high temperature ($\tau = 1$) and anneals it over training iterations may improve the quality of policy targets in later training stages when the network has already converged to strong play.

7 Related Work

The original AlphaGo [3] combined supervised learning from human games with MCTS and self-play. AlphaGo Zero [2] removed the supervised pre-training, demonstrating that self-play alone was sufficient. AlphaZero [1] generalised this approach to Chess and Shogi using a single unified algorithm. MuZero [4] further extended the paradigm by learning a model of the environment’s dynamics rather than relying on a perfect simulator.

Open-source reimplementations include `alpha-zero-general` [7], which provides a clean Python framework for multiple games. AlphaMinus distinguishes itself by being entirely self-contained in PyTorch with no external game framework dependencies.

8 Conclusion

We have presented AlphaMinus, a clean and fully functional PyTorch reimplementation of AlphaZero applied to Connect4. The system achieves strong performance from pure self-play within 30 training iterations, demonstrating the effectiveness of the AlphaZero paradigm even at small scale. We hope this codebase serves as a useful reference implementation for researchers and practitioners interested in deep reinforcement learning and game playing.

References

- [1] D. Silver et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.
- [2] D. Silver et al. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.
- [3] D. Silver et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [4] J. Schrittwieser et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.
- [5] R. Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In *International Conference on Computers and Games*, 2006.
- [6] L. Kocsis and C. Szepesvári. Bandit based Monte-Carlo planning. In *European Conference on Machine Learning*, 2006.
- [7] S. Thakoor, S. Nair, and M. Jhunjhunwala. Learning to play Othello without human knowledge. Technical report, Stanford University, 2016.